

Name:Manav Mangesh Uttekar

Roll no.: 71

Problem statement: There are flight paths between cities. If there is a flight between city A and city B then there is an edge between the cities. The cost of the edge can be the time that flight take to reach city B from A, or the amount of fuel used for the journey. Represent this as a graph. The node can be represented by airport name or name of the city. Use adjacency list representation of the graph or use adjacency matrix representation of the graph. Check whether the graph is connected or not. Justify the storage representation used.

```
from collections import defaultdict, deque

class FlightGraph:
    def __init__(self):
        self.adj = defaultdict(list) # city -> list of (neighbor, time)

    def add_flight(self, city_from, city_to, time):
        self.adj[city_from].append((city_to, time))
        self.adj[city_to].append((city_from, time)) # undirected graph

    def display(self):
        print("Adjacency List Representation of Flight Graph:")
        for city in self.adj:
            connections = ', '.join([f"({dest}, time={time})" for dest,
time in self.adj[city]])
            print(f"{city} -> {connections}")

    def is_connected(self):
        if not self.adj:
            return True

        visited = set()
        queue = deque()

        start = next(iter(self.adj)) # Get any starting city
        queue.append(start)
        visited.add(start)

        while queue:
            current = queue.popleft()
            for neighbor, _ in self.adj[current]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)

        return len(visited) == len(self.adj)

# Example usage
```

```
graph = FlightGraph()
graph.add_flight("Mumbai", "Delhi", 120)
graph.add_flight("Delhi", "Bangalore", 180)
graph.add_flight("Mumbai", "Chennai", 150)
graph.add_flight("Chennai", "Kolkata", 200)

graph.display()

if graph.is_connected():
    print("\n The flight graph is CONNECTED.")
else:
    print("\n The flight graph is NOT connected.")
```

OUTPUT:

```
Adjacency List Representation of Flight Graph:
Mumbai -> (Delhi, time=120), (Chennai, time=150)
Delhi -> (Mumbai, time=120), (Bangalore, time=180)
Bangalore -> (Delhi, time=180)
Chennai -> (Mumbai, time=150), (Kolkata, time=200)
Kolkata -> (Chennai, time=200)

✅ The flight graph is CONNECTED.
```